

Rabbit R1 "Creations" Development Specification

1. Platform Overview The Rabbit R1 runs on RabbitOS, which is built upon an Android Open Source Project (AOSP) base. A "Creation" is a specialized, lightweight web-based mini-application running within a sandboxed Android WebView environment via Flutter. The goal is to build an ultra-compact web application (HTML/CSS/JS) that interacts with the native OS through a specific bridge.

GitHub

2. Hardware & Environmental Constraints You must adhere strictly to these physical and environmental limitations when designing the UI and interaction logic:

GitHub

- **Viewport & Resolution:** The physical display is a 2.88" touchscreen with a 480x640 resolution. However, the software viewport for Creations is strictly restricted to **240x282 pixels in portrait mode**. The UI must not require horizontal scrolling.

GitHub

- **Input Methods:**

GitHub

- **Touchscreen:** Standard tap interactions are supported.

GitHub

- **Scroll Wheel:** The wheel only rotates (it does not click) and sends standard `KEY_UP` and `KEY_DOWN` (DPAD navigation) events. Ensure your UI supports scrolling lists via these key events.

GitHub

- **Physical Button:** There is a single side button that acts as both Push-To-Talk (PTT) and Power.

GitHub

- **Audio & Hardware:** The device has a camera, microphone, and speakers, but accessing them requires routing requests through the specific Flutter bridge rather than standard Web APIs.

3. Software Architecture & The Flutter Bridge Since the Creation is a WebView running inside a Flutter application, it cannot directly interface with Android hardware APIs.

- **The Bridge (`window.flutter_inappwebview`):** All hardware interactions and OS-level requests (like utilizing the camera, accessing the accelerometer, or processing speech-to-text) must be bound to this specific JavaScript object.
- **No Native Dependencies:** Do not use heavy frameworks (like React, Angular, or Next.js) unless specifically requested. Plain Vanilla JavaScript (or a lightweight bundler like Vite) is required to ensure optimal performance.

4. Data Persistence Strategy The R1 Creation environment treats local data as ephemeral.

- **Strict Rule:** Do not rely on `localStorage`, `sessionStorage`, `IndexedDB`, or native file system access for long-term data persistence.
- **External Database:** All persistent data (like user preferences, logs, or state) must be handled via standard JavaScript `fetch` API calls connecting to an external REST API or serverless backend (e.g., Supabase, Firebase, or a custom Node.js backend). The AI assistant must implement asynchronous `fetch` requests for reading and writing data.

5. Required Project Structure (Vite) The codebase should follow this scaffold:

- `package.json` & `vite.config.js`: Build and bundling configuration.
- `src/main.js`: The primary entry point for logic and DOM manipulation.
- `src/style.css`: Global styles optimized for the 240x282 display.
- `src/lib/flutter-channel.js`: The dedicated module for binding JavaScript functions to `window.flutter_inappwebview`.
- `src/lib/device-controls.js`: Logic for interpreting wheel scrolls (`KEY_UP` / `KEY_DOWN`) and touch events.
- `src/lib/ui-design.js`: DOM construction and visual component rendering.

6. Task Objective *Review the user's specific application goal provided in the chat. Write clean, modular, and heavily commented code that respects the 240x282 constraint, utilizes `fetch` for external data persistence, and leverages the Flutter bridge for any necessary hardware interactions.*

To prevent PowerShell from sneaking a Byte Order Mark (BOM) into your files in the future, you need to change how you handle file writing.

This is the method I used to fix your script. It works perfectly on **all** versions of PowerShell (Windows and Linux) because it uses the underlying .NET framework to explicitly turn off the BOM.

Instead of `Out-File` or `>`, use this pattern:

PowerShell

```
# Define the encoder once at the top of your script
$utf8NoBom = New-Object System.Text.UTF8Encoding($false)

# Define your file content
$content = '{"name": "my-app"}'
```

```
# Write the file cleanly  
[System.IO.File]::WriteAllText("path/to/file.json", $content, $utf8NoBom)
```

Zip final product to upload.